

MVP SILVER ECONOMY 2026.2

POLÍTICA DE GOVERNANÇA

InformAju • Versão 1.0

Governança de produto, equipe, desenvolvimento, código e uso interno de Inteligência Artificial.

Instituição	UNINASSAU Aracaju
Curso	Análise e Desenvolvimento de Sistemas (ADS)
Projeto	InformAju
Responsável	Gabriel David Vacirca - PO, Scrum Master, líder e desenvolvedor
Data	29 de setembro de 2026

INFORMAJU — O GUIA CONFIÁVEL

Controle do documento

Título	Política de Governança do Projeto InformAju
Versão	1.0
Status	Aprovada para uso inicial do projeto
Proprietário do documento	Product Owner / Líder do Projeto
Periodicidade de revisão	Sempre que houver mudança material de processo, arquitetura, responsabilidades ou ambiente

Histórico de versões

Versão	Data	Descrição	Responsável
1.0	29/09/2026	Emissão inicial da política de governança do InformAju.	Gabriel D. Vacirca

Identificação da equipe

Integrante	Matrícula	Papel principal
Gabriel David Vacirca	16035571	PO, Scrum Master, líder e desenvolvedor
Jorge Felipe Trindade Mendonça	16035609	Desenvolvedor
Yuri Cruz Brandão	16036094	Desenvolvedor
Felipe Gabriel dos Santos Vasconcelos	16036165	Desenvolvedor

Base institucional

Este documento formaliza a governança inicial do InformAju e deverá ser atualizado conforme o projeto evoluir. Ele não antecipa decisões arquiteturais ou técnicas que ainda não foram oficialmente aprovadas pela equipe.

Sumário

1. Objetivo e escopo
 2. Princípios de governança
 3. Papéis e responsabilidades
 4. Governança da visão do produto e Lean Inception
 5. Processo de tomada de decisão
 6. Gestão ágil com Scrum
 7. Governança do código: GitFlow e Conventional Commits
 8. Pull Requests e revisão de código
 9. Ambiente padronizado de desenvolvimento
 10. Governança do uso de IA pela equipe
 11. Estado atual da arquitetura e tecnologias
 12. Segurança, credenciais e dados
 13. Rastreabilidade e evidências de participação
 14. Gestão de mudanças e evolução da política
 15. Aprovação e vigência
- Apêndice A. Resumo para a Landing Page

1. Objetivo e escopo

Esta Política de Governança estabelece as regras iniciais de organização, decisão, desenvolvimento, controle de código, padronização do ambiente e uso interno de ferramentas de Inteligência Artificial pela equipe do InformAju.

Seu objetivo é garantir alinhamento entre os integrantes, consistência nas entregas, rastreabilidade das decisões e evolução controlada do MVP. A política se aplica à equipe do projeto, aos repositórios oficiais, aos artefatos de planejamento e aos ambientes de desenvolvimento utilizados no InformAju.

A versão 1.0 é deliberadamente evolutiva: decisões ainda abertas, como a arquitetura definitiva do software, não são tratadas como concluídas. Quando forem definidas, deverão ser incorporadas em nova versão desta política ou em documentos técnicos complementares.

2. Princípios de governança

- **Transparência:** decisões relevantes devem ser compreensíveis, registráveis e acessíveis à equipe.
- **Colaboração crítica:** propostas devem ser discutidas com abertura a contrapontos antes da decisão.

- Responsabilidade individual e coletiva: cada integrante responde pela qualidade do que produz e pela contribuição para o objetivo comum.
- Rastreabilidade: mudanças relevantes devem deixar evidências em backlog, documentação, commits, Pull Requests ou registros equivalentes.
- Consistência: código, documentação, ambientes e agentes de IA devem seguir padrões comuns do projeto.
- Evolução controlada: mudanças de escopo, processo ou tecnologia devem ser avaliadas antes de serem incorporadas.

3. Papéis e responsabilidades

3.1 Product Owner (PO)

Gabriel David Vacirca exerce o papel de Product Owner e é responsável por preservar a visão do produto, organizar e priorizar o Product Backlog, apoiar decisões de escopo e garantir coerência entre as entregas e os objetivos do InformAju.

- priorizar backlog e objetivos de curto prazo;
- mediar decisões relacionadas a escopo e prioridade;
- validar o alinhamento das entregas com a visão do produto;
- realizar a revisão final dos Pull Requests da equipe;
- manter e distribuir o contexto oficial utilizado pelos agentes de IA;
- preparar e manter o ambiente-base de desenvolvimento.

3.2 Scrum Master

O papel de Scrum Master também é exercido por Gabriel David Vacirca. Sua responsabilidade é facilitar o uso do Scrum, acompanhar o andamento das sprints, apoiar a remoção de impedimentos e promover a disciplina necessária para que o processo ágil seja útil à equipe.

3.3 Desenvolvedores

Os quatro integrantes atuam diretamente no desenvolvimento. Cada desenvolvedor é responsável pelas tarefas que assumir, pela revisão prévia do próprio código, pela documentação pertinente e pela comunicação de impedimentos ou riscos identificados.

Responsabilidade sobre código assistido por IA

O uso de IA não transfere responsabilidade. O integrante que utilizar uma ferramenta de IA continua responsável por compreender, revisar, testar e justificar o código ou conteúdo incorporado ao projeto.

4. Governança da visão do produto e Lean Inception

A construção da visão e do escopo inicial do InformAju utiliza práticas de Lean Inception. O workshop é empregado para transformar a visão do problema em decisões progressivamente mais concretas sobre público, personas, jornadas, funcionalidades, priorização, MVP e roadmap.

Os artefatos produzidos nessa etapa são considerados referências de decisão do produto. Mudanças posteriores que afetem escopo ou proposta de valor devem considerar os impactos sobre esses artefatos e ser registradas quando relevantes.

5. Processo de tomada de decisão

As decisões relevantes serão precedidas por discussão crítica entre os quatro integrantes. O objetivo é avaliar diferentes perspectivas antes da escolha, evitando decisões individuais sem oportunidade de contraponto.

Quando não houver consenso, a decisão será submetida à votação. Cada integrante possui um voto. Em caso de empate, o Product Owner terá a responsabilidade pela decisão final, considerando a visão do produto, as necessidades do público-alvo, o prazo acadêmico, o roadmap, a viabilidade técnica e o impacto no MVP.

Regra de decisão

Discussão crítica → busca de consenso → votação → em caso de empate, decisão final do PO.

6. Gestão ágil com Scrum

Após a consolidação da visão e do escopo inicial, o desenvolvimento será organizado com Scrum e dividido em sprints. A quantidade e a duração das sprints ainda não estão definidas nesta versão do documento e serão estabelecidas após a conclusão do planejamento inicial e do roadmap do MVP.

As funcionalidades e atividades serão mantidas em backlog priorizado. Uma nova funcionalidade ou mudança significativa de escopo deverá ser registrada e avaliada antes de entrar em uma sprint, evitando alterações informais que comprometam previsibilidade e rastreabilidade.

A equipe poderá adaptar a cerimônia e o nível de formalidade do Scrum à escala acadêmica do projeto, desde que preserve os princípios de planejamento, acompanhamento, revisão e melhoria contínua.

7. Governança do código: GitFlow e Conventional Commits

7.1 Repositório oficial

O código-fonte do InformAju será mantido em repositório oficial Git/GitHub. O repositório será a referência para versões do software, histórico de alterações, colaboração e evidências técnicas de participação.

7.2 Estratégia de branches com GitFlow

A equipe adotará uma variação simplificada do GitFlow, adequada ao porte do MVP acadêmico. A estrutura-base será:

- main - versões estáveis e entregáveis;
- develop - integração do desenvolvimento;
- feature/* - novas funcionalidades;
- bugfix/* - correções realizadas durante o desenvolvimento;
- release/* - preparação de uma versão, quando necessário;
- hotfix/* - correções urgentes em versão estável.

```
feature/F01-encontrar-servico  
bugfix/corrige-menu-mobile  
release/v0.1.0  
hotfix/corrige-fonte-servico
```

Push direto em main deverá ser evitado. A integração de mudanças relevantes ocorrerá por Pull Request. Branches de release e hotfix poderão ser usadas somente quando fizerem sentido para o estágio do MVP, evitando burocracia desnecessária.

7.3 Padrão de commits

As mensagens de commit seguirão Conventional Commits, permitindo histórico legível e classificação clara das mudanças.

```
feat(F01): adiciona busca por necessidade  
fix(header): corrige navegação no mobile  
docs: atualiza política de governança  
refactor(servicos): reorganiza componente de detalhes  
test: adiciona validação da busca  
chore: atualiza dependências
```

Commits devem representar alterações coerentes. Mensagens genéricas como “mudanças”, “teste”, “coisas” ou “final” deverão ser evitadas.

8. Pull Requests e revisão de código

Cada desenvolvedor deverá revisar seu próprio código antes de abrir um Pull Request. Após o self-review, a branch será enviada ao repositório e o Pull Request será aberto com uma descrição objetiva do que foi alterado e, quando aplicável, referência à tarefa ou feature correspondente.

O fluxo padrão será:

Fluxo de integração

Desenvolvimento → self-review → push da branch → Pull Request → revisão do PO → ajustes, se necessários → aprovação → merge.

A revisão final dos Pull Requests dos demais desenvolvedores é responsabilidade do PO/líder. A revisão deve considerar, quando aplicável:

- funcionamento esperado;
- compatibilidade com escopo e arquitetura vigente;
- clareza e organização do código;
- padrões de projeto e convenções da equipe;
- riscos de regressão;
- ausência de segredos ou credenciais no código;
- testes ou validações pertinentes;
- documentação necessária.

Como controle mínimo de segregação, Pull Requests criados pelo próprio PO deverão receber revisão de pelo menos um dos outros desenvolvedores antes do merge.

9. Ambiente padronizado de desenvolvimento

O ambiente-base de desenvolvimento será preparado e documentado pelo PO. Todos os integrantes deverão utilizar a configuração oficial do projeto para reduzir divergências de versão, dependências e comportamento entre máquinas.

A documentação do ambiente deverá, progressivamente, registrar:

- versões das principais ferramentas;
- dependências e comandos de instalação;
- estrutura inicial dos projetos;
- scripts de execução e build;
- variáveis de ambiente exigidas;
- convenções de código;
- instruções para executar e validar a aplicação localmente.

Quando o PO atualizar uma configuração obrigatória, a mudança deverá ser comunicada e os demais desenvolvedores deverão atualizar seus ambientes antes de continuar atividades impactadas pela alteração.

10. Governança do uso de IA pela equipe

Esta seção trata exclusivamente do uso de ferramentas e agentes de Inteligência Artificial pelos integrantes da equipe. A funcionalidade de IA do produto InformAju será detalhada em documento próprio.

10.1 Contexto oficial compartilhado

O PO manterá um Contexto Master oficial do InformAju para uso nos agentes de IA. Esse contexto deverá concentrar informações relevantes e aprovadas do projeto, como visão do produto, público, funcionalidades, restrições, decisões, padrões, arquitetura quando definida e demais regras necessárias para orientar o trabalho assistido por IA.

Todos os integrantes deverão utilizar o contexto oficial em seus respectivos agentes. Quando o Contexto Master for atualizado, os desenvolvedores deverão atualizar também seus agentes antes de utilizar a IA em atividades impactadas pela nova versão.

Objetivo

Evitar que agentes diferentes trabalhem com versões conflitantes do projeto e reduzir inconsistências entre decisões, código e documentação.

10.2 Credenciais e acesso

Quando ferramentas, APIs ou ambientes exigirem chaves, tokens ou outras credenciais, o acesso será disponibilizado e administrado conforme orientação do PO. Credenciais reais não poderão ser armazenadas diretamente no código-fonte nem versionadas no Git.

Sempre que aplicável, deverão ser utilizadas variáveis de ambiente ou mecanismos equivalentes. Arquivos de exemplo, como `.env.example`, poderão documentar os nomes das variáveis necessárias sem conter valores secretos.

10.3 Validação humana

Código, documentação ou decisões sugeridas por IA não serão considerados automaticamente corretos. O integrante que utilizar a ferramenta deverá compreender o resultado, revisar o conteúdo, validar sua adequação ao projeto e testar o comportamento quando houver código executável.

11. Estado atual da arquitetura e tecnologias

Nesta versão 1.0, a arquitetura definitiva do software InformAju ainda não foi formalmente definida. A equipe decidiu não registrar como concluídas escolhas de camadas, módulos, APIs, padrões arquiteturais ou desenho de integração que ainda estão em avaliação.

11.1 Estado atual

A Landing Page é o primeiro artefato Web implementado. Ela utiliza React com Vite e está versionada/publicada por meio da infraestrutura associada ao repositório GitHub do projeto.

11.2 Restrições tecnológicas conhecidas

A implementação do MVP deverá respeitar a stack obrigatória do projeto acadêmico. Entre as tecnologias já determinadas estão React para Web e Java com Spring Boot e Hibernate no back-end. A escolha do banco relacional e do framework Mobile será formalizada posteriormente entre as opções permitidas pelo edital.

A arquitetura técnica será documentada quando definida, respeitando também os critérios acadêmicos de padrões de projeto e arquitetura estabelecidos para a avaliação final.

Distinção importante

Stack obrigatória não significa arquitetura já definida. A política registra as tecnologias impostas pelo projeto acadêmico sem antecipar o desenho arquitetural que a equipe ainda irá escolher.

12. Segurança, credenciais e dados

- segredos, tokens e chaves não devem ser commitados no repositório;
- variáveis sensíveis devem ser carregadas por ambiente seguro e documentadas apenas por nome;
- acessos compartilhados devem ser concedidos somente aos integrantes que necessitem deles;
- qualquer exposição acidental de credencial deve ser comunicada imediatamente e a credencial deve ser revogada/rotacionada;
- dados utilizados pelo projeto devem ser limitados ao necessário para o funcionamento e validação do MVP.

13. Rastreabilidade e evidências de participação

A participação dos integrantes será demonstrável por evidências técnicas e organizacionais produzidas ao longo do projeto.

- commits no repositório;

- Pull Requests e revisões de código;
- tarefas assumidas e movimentadas na ferramenta de gestão;
- documentação produzida;
- participação em decisões, revisões e apresentações;
- entregas associadas às sprints e ao roadmap.

A equipe deverá evitar concentrar todas as contribuições em um único integrante, mantendo divisão de trabalho compatível com os papéis e atividades acordados.

14. Gestão de mudanças e evolução da política

Esta política é um documento vivo. Mudanças relevantes devem seguir o mesmo princípio de governança utilizado no restante do projeto: proposta, discussão, decisão, registro e comunicação.

Uma nova versão deverá ser emitida quando houver mudança material em papéis, processo de revisão, arquitetura, estratégia de branches, ambiente oficial, governança de IA ou outra regra que afete de forma significativa o trabalho da equipe.

14.1 Decisões ainda abertas

- arquitetura oficial do software;
- quantidade e duração das sprints;
- banco de dados relacional escolhido;
- tecnologia Mobile escolhida entre as opções permitidas;
- estratégia completa de testes e automação;
- processo de CI/CD e releases;
- documento específico da funcionalidade de IA do produto.

15. Aprovação e vigência

A Política de Governança v1.0 entra em vigor a partir de sua aprovação pela equipe e permanece válida até ser substituída por versão posterior.

A adoção desta política representa o compromisso dos integrantes em seguir os processos aqui definidos, comunicar desvios relevantes e contribuir para a evolução responsável do InformAju.

Integrante	Matrícula	Ciência / aprovação
Gabriel David Vacirca	16035571	_____
Jorge Felipe Trindade Mendonça	16035609	_____
Yuri Cruz Brandão	16036094	_____
Felipe Gabriel dos Santos Vasconcelos	16036165	_____

Apêndice A. Resumo para a Landing Page

Uso sugerido

Este resumo pode ser utilizado na seção “Projeto e governança” da Landing Page. Ele não substitui a política completa.

O InformAju é desenvolvido por Gabriel David Vacirca, Jorge Felipe Trindade Mendonça, Yuri Cruz Brandão e Felipe Gabriel dos Santos Vasconcelos. Gabriel atua como Product Owner, Scrum Master, líder e desenvolvedor, enquanto todos os integrantes participam do desenvolvimento do projeto.

A visão do produto é construída com práticas de Lean Inception e as decisões relevantes são discutidas coletivamente. Quando não há consenso, a equipe realiza votação; em caso de empate, o Product Owner toma a decisão final considerando a visão, as prioridades e as restrições do MVP.

O desenvolvimento será organizado com Scrum e sprints. O código é versionado com Git/GitHub, seguindo GitFlow e Conventional Commits. Alterações relevantes são submetidas por Pull Request, com revisão final do PO. A equipe também utiliza um ambiente e um Contexto Master padronizados para manter consistência no trabalho e no uso de ferramentas de IA.